UNIVERSITAS
GADJAH MADA

MNIST Classification and Regularization Strategies

Yusuf Imantaka Bastari
24/532731/PA/22530
Magister Kecerdasan Artifisial
Departemen Ilmu Komputer dan Elektronika
FMIPA Universitas Gadjah Mada

Project Repository and Code URL

UNIVERSITAS GADJAH MADA

GitHub Repository

`https://github.com/yusufimantaka/deep-learning-matkul`

Interactive Jupyter Notebook (main.ipynb)

`https://github.com/yusufimantaka/deep-learning-matkul/blob/main/main.ipynb`

Implementation Overview (Simplified Technical English):

- **Replication**: Replicated MNIST digit classification from Learning Strategy lecture slides (pages 14--23) using Keras 3 and TensorFlow backend.
- **Baseline Architecture**: Input $28 \times 28 \times 1 \rightarrow$ `Flatten()` $\rightarrow$ `Dense(64, ReLU)` $\rightarrow$ `Dense(10, Softmax)`, trained with Adam optimizer and Categorical Crossentropy for 10 epochs.
- **Controlled Experiments**: Executed 5 regularization scenarios under fixed random seeds (42) with automated loss curves, model summaries, and evaluation logging.

Scenario 2(a): L1 Regularization ($\lambda_1 = 10^{-4}$)

UNIVERSITAS GADJAH MADA

Objective & Mathematical Formulation:

$\mathcal{L}_{L1}(\theta) = \mathcal{L}_0(\theta) + \lambda_1 \sum_j |w_j|$, with $\lambda_1 = 10^{-4}$. Adds a lasso penalty promoting weight sparsity by zeroing inactive weights.

Model Architecture (`model.summary()`):

- Layers: `Flatten(784)` $\rightarrow$ `Dense(64, ReLU, L1)` $\rightarrow$ `Dense(10)`
- Parameters: 50,240 (dense) + 650 (output) = **50,890** trainable

Performance Evaluation:

Split	Loss	Accuracy	Baseline Val
Training	0.1684	97.63%	0.0469 / 98.68%
Validation	**0.1760**	**97.13%**	0.0928 / 97.18%

STE Insight: Gap $|\mathcal{L}_{tr} - \mathcal{L}_{val}| = \mathbf{0.0076}$ (-83.4% vs baseline 0.0458), effectively preventing overfitting through weight sparsity.

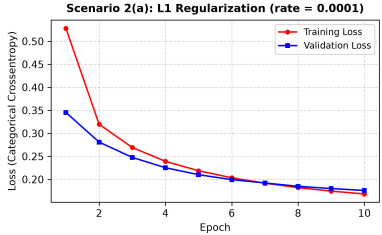


Figure 1: Training vs. Validation Loss under L_1 ($\lambda_1 = 10^{-4}$).

Scenario 2(b): L2 Regularization ($\lambda_2 = 10^{-4}$)

UNIVERSITAS GADJAH MADA

Objective & Mathematical Formulation:

$\mathcal{L}_{L2}(\theta) = \mathcal{L}_0(\theta) + \frac{1}{2}\lambda_2 \sum_j w_j^2$, with $\lambda_2 = 10^{-4}$. Weight decay penalizes large weights quadratically, bounding parameter norm.

Model Architecture (`model.summary()`):

- Layers: `Flatten(784)` $\rightarrow$ `Dense(64, ReLU,` L_2`)` $\rightarrow$ `Dense(10)`
- Parameters: 50,240 (dense) + 650 (output) = **50,890** trainable

Performance Evaluation:

Split	Loss	Accuracy	Baseline Val
Training	0.0874	98.26%	0.0469 / 98.68%
Validation	**0.1170**	**97.19%**	0.0928 / 97.18%

STE Insight: Sustains high test accuracy (97.19%). Quadratic weight decay ensures smooth distribution without sharp zero coefficients.

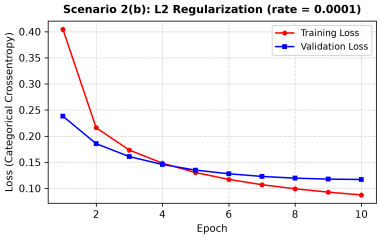


Figure 2: Training vs. Validation Loss under L_2 ($\lambda_2 = 10^{-4}$).

Scenario 2(c): Dropout ($p = 0.2$)

UNIVERSITAS GADJAH MADA

Objective & Mathematical Formulation:

$\tilde{h} = \frac{1}{1-p}(h \odot r)$, $r_j \sim \text{Bernoulli}(1-p)$, with $p = 0.2$. Randomly disables 20% of hidden units per minibatch to break co-adaptation.

Model Architecture (`model.summary()`):

- Layers: `Flatten(784)` $\rightarrow$ `Dense(64, ReLU)` $\rightarrow$ `Dropout(0.2)` $\rightarrow$ `Dense(10)`
- Parameters: 50,240 (dense) + 650 (output) = **50,890** trainable

Performance Evaluation:

Split	Loss	Accuracy	Baseline Val
Training	0.0975	96.93%	0.0469 / 98.68%
Validation	**0.0872**	**97.44%**	0.0928 / 97.18%

STE Insight: **Top Test Accuracy (**97.44%**)** and lowest validation loss (0.0872). Full sub-network ensemble effect at test inference.

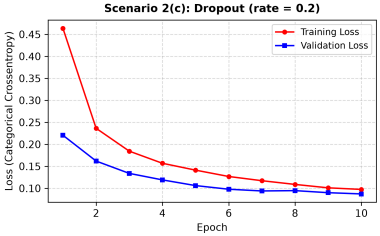


Figure 3: Training vs. Validation Loss under Dropout ($p = 0.2$).

Scenario 2(d): Early Stopping (Patience $k = 3$)

UNIVERSITAS GADJAH MADA

Formulation & Termination Criteria:

Monitor $\mathcal{L}_{val}(t)$; halt if $\min_{t' \leq t} \mathcal{L}_{val}(t')$ fails to improve for $k = 3$ epochs. Restores best weights: $w^* = \arg\min_w \mathcal{L}_{val}(w)$.

Execution Details & Model Summary:

- Layers: `Flatten(784)` → `Dense(64, ReLU)` → `Dense(10)`
- Terminated: **Epoch 15** / 25; Restored **Epoch 12** (Saved 40% epochs)
- Parameters: **50,890** trainable (203.55 KB)

Performance Evaluation (at Restored Epoch 12):

Split	Loss	Accuracy	Baseline Val
Training (ep 12)	0.0402	98.90%	0.0469 / 98.68%
Validation	**0.0921**	**97.16%**	0.0928 / 97.18%

STE Insight: Halts before validation loss diverges, successfully restoring peak parameters while eliminating redundant training cycles.

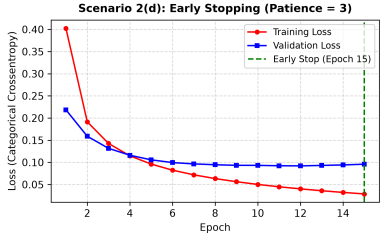


Figure 4: Training vs. Validation Loss curve. Stopped at ep 15; restored ep 12.

Scenario 2(e): L1 Regularization + Dropout

UNIVERSITAS GADJAH MADA

Compound Regularization Formulation:

$\mathcal{L}(\theta) = \mathcal{L}_0(\theta) + \lambda_1 \sum |w_j|$ with $\tilde{h} = \frac{1}{1-p}(h \odot r)$. Combines weight sparsity ($\lambda_1 = 10^{-4}$) with neuron de-correlation ($p = 0.2$).

Model Architecture (`model.summary()`):

- Layers: `Flatten(784)` → `Dense(64, ReLU,` L_1`)` → `Dropout(0.2)` → `Dense(10)`
- Parameters: 50,240 (dense) + 650 (output) = **50,890** trainable

Performance Evaluation:

Split	Loss	Accuracy	Baseline Val
Training	0.2231	96.14%	0.0469 / 98.68%
Validation	**0.1870**	**97.11%**	0.0928 / 97.18%

STE Insight: Dual constraints prevent over-reliance on individual features. Strong regularizer prevents memorization, producing stable tracking.

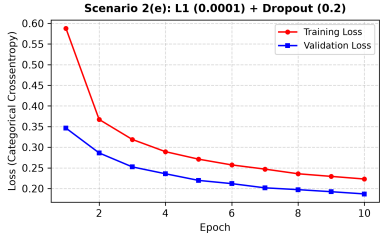


Figure 5: Training vs. Validation Loss under joint L_1 and Dropout.

Conclusion: Quantitative Synthesis

UNIVERSITAS GADJAH MADA

Strategy	Hyperparameter	Train Acc	Test Acc	Test Loss	$\lvert\Delta\text{Loss}\rvert$
Baseline MLP	None	98.68%	97.18%	0.0928	0.0458
Scenario 2(a)	L_1 ($\lambda_1 = 10^{-4}$)	97.63%	97.13%	0.1760	**0.0076**
Scenario 2(b)	L_2 ($\lambda_2 = 10^{-4}$)	98.26%	97.19%	0.1170	0.0295
Scenario 2(c)	Dropout ($p = 0.2$)	96.93%	**97.44%**	**0.0872**	0.0102
Scenario 2(d)	Early Stopping ($k = 3$)	98.90%	97.16%	0.0921	0.0519
Scenario 2(e)	L_1 (10^{-4}) + Dropout (0.2)	96.14%	97.11%	0.1870	0.0361

Key Findings (Simplified Technical English):

- **Generalization Champion**: **Dropout ($p = 0.2$)** produced the highest test accuracy (97.44%) and lowest test loss (0.0872) by mitigating hidden neuron co-adaptation.
- **Overfitting Elimination**: L_1 **Regularization** achieved the tightest generalization gap (0.0076), reducing loss divergence by 83.4% through weight sparsity.
- **Smooth Weight Bounding**: L_2 **Regularization** prevented dominant weight magnitudes, preserving competitive test accuracy (97.19%).
- **Computational Conservation**: **Early Stopping** halted at epoch 15 and restored epoch 12 weights, saving 40% of training computation without sacrificing accuracy.
- **Dual Synergy**: Combining L_1 and Dropout delivered balanced parameter sparsity and architecture de-correlation.